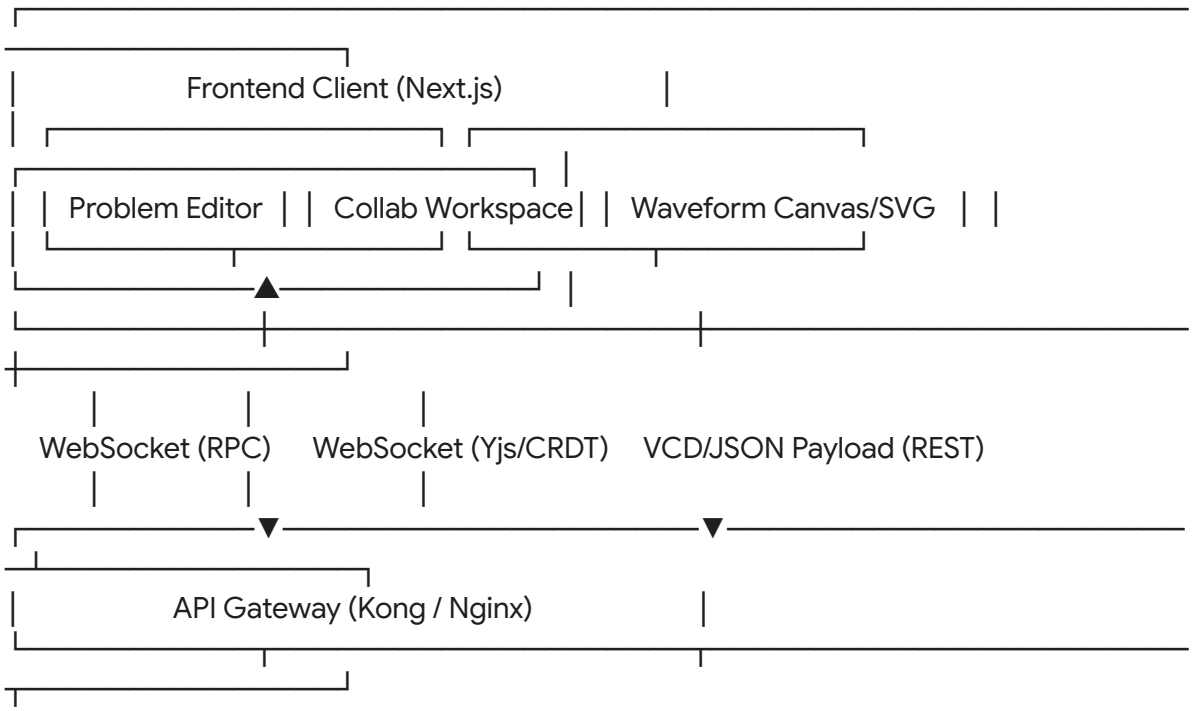
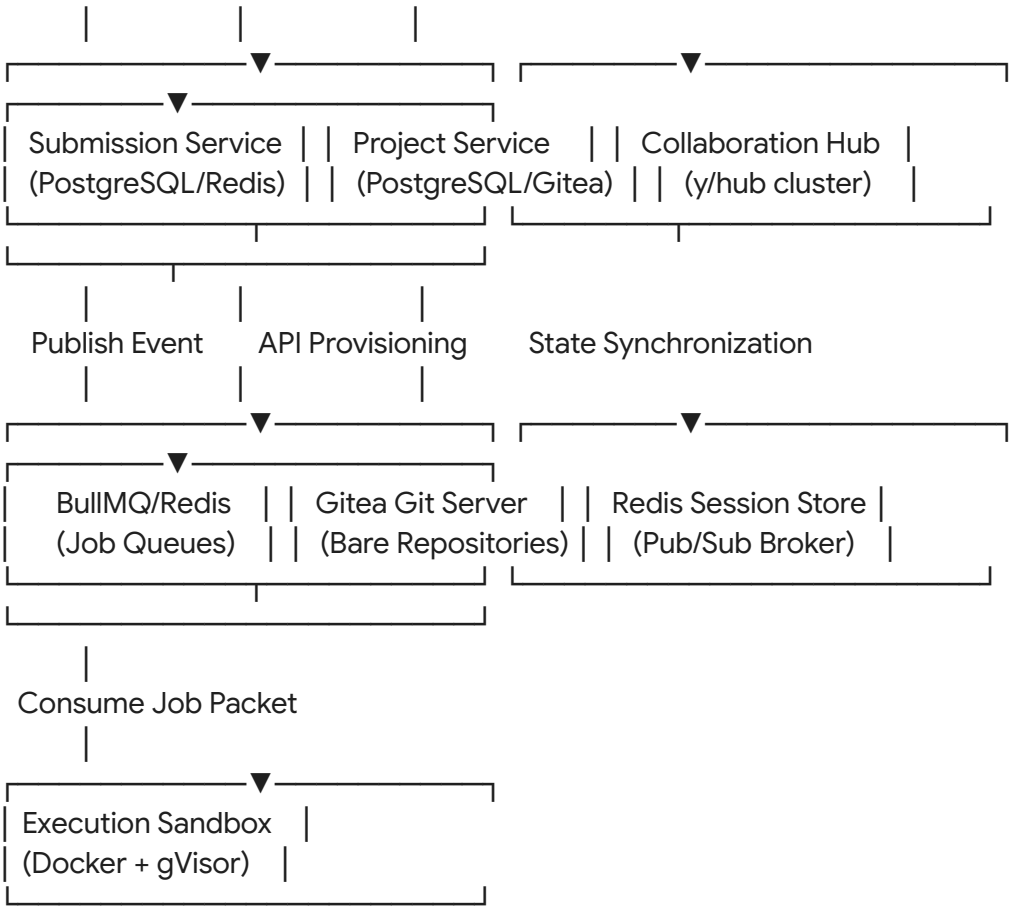
Architectural Integration Plan: Merging chipdev.io Practice Corpus into the bitforbytes Platform

The strategic combination of the *bitforbytes* ecosystem and the interactive hardware design capabilities of *chipdev.io* establishes a single web-native system for hardware design, verification, and engineering evaluation[1]. This technical document outlines the integration of the open-source repository KnowAashish/chipdev.io into a microservices-based web application[1]. It details the design of a highly isolated execution engine, a real-time collaborative workspace, and an automated verification system.

High-Level System Architecture and Design Patterns

The platform is designed to decouple user-facing interactions from resource-heavy compilation and simulation workloads. It uses a three-tier design to ensure that heavy hardware compilation runs do not degrade editor responsiveness or delay concurrent user updates[2].





The system uses three primary interfaces to optimize state distribution and keep interactions highly responsive:

- **REST/GraphQL Gateway**: Handles typical transactions like problem queries, submission history, profile management, and repository metadata updates.
- **Persistent WebSockets (JSON-RPC)**: Streams standard terminal outputs (stdout, stderr) and compilation status changes directly to the client during long-running simulations[5].
- **Collaborative WebSocket Provider (Yjs Protocol)**: Uses highly optimized WebSocket servers backed by Redis Pub/Sub to synchronize multi-user cursors, text edits, and file trees across teams in real time[4].

Transaction records, profile analytics, and repository states are stored in a primary PostgreSQL database. Transient application states—including user sessions, active compilation queues, and collaborative sync states—are managed through a Redis cluster[6]. Long-running compile and test tasks are published to a Redis-backed BullMQ queue[8]. Highly isolated workers pull jobs from this queue and execute them within sandboxed environments secured by the gVisor container runtime, protecting the host system from potential security exploits[10].

Core Database Schema and Entity Relations

The relational database schema is structured to maintain transactional integrity across user

profiles, challenges, submissions, and repositories.

```
-- PostgreSQL Database Schema Definitions

CREATE TYPE challenge_difficulty AS ENUM ('easy', 'medium', 'hard');
CREATE TYPE run_status AS ENUM ('pending', 'compiling', 'completed', 'failed',
'compilation_error', 'runtime_error', 'timeout');
CREATE TYPE member_role AS ENUM ('owner', 'maintainer', 'contributor', 'viewer');

CREATE TABLE users (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    username VARCHAR(50) UNIQUE NOT NULL,
    email VARCHAR(255) UNIQUE NOT NULL,
    password_hash VARCHAR(255) NOT NULL,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE user_profiles (
    user_id UUID PRIMARY KEY REFERENCES users(id) ON DELETE CASCADE,
    first_name VARCHAR(100),
    last_name VARCHAR(100),
    avatar_url TEXT,
    skills_inventory VARCHAR(50)[] DEFAULT '{}',
    completed_challenges_count INT DEFAULT 0,
    normalized_readiness_score NUMERIC(5,2) DEFAULT 0.00
);

CREATE TABLE challenges (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    title VARCHAR(255) NOT NULL,
    slug VARCHAR(255) UNIQUE NOT NULL,
    description_raw TEXT NOT NULL,
    difficulty challenge_difficulty NOT NULL,
    labels VARCHAR(50)[] DEFAULT '{}',
    skeleton_code TEXT NOT NULL,
    public_testbench_id UUID,
```

```
    hidden_testbench_id UUID NOT NULL,
    execution_timeout_ms INT DEFAULT 15000,
    allocation_memory_mb INT DEFAULT 512,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE user_submissions (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id UUID REFERENCES users(id) ON DELETE CASCADE,
    challenge_id UUID REFERENCES challenges(id) ON DELETE CASCADE,
    code_payload TEXT NOT NULL,
    status run_status DEFAULT 'pending',
    simulation_logs TEXT,
    waveform_dump_url TEXT,
    cpu_run_time_ms INT,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE git_repositories (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    project_name VARCHAR(100) NOT NULL,
    details TEXT,
    creator_id UUID REFERENCES users(id) ON DELETE CASCADE,
    gitea_clone_link TEXT NOT NULL,
    manifest_payload JSONB DEFAULT '{}'::jsonb,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE repository_access_control (
    repository_id UUID REFERENCES git_repositories(id) ON DELETE CASCADE,
    user_id UUID REFERENCES users(id) ON DELETE CASCADE,
    role_level member_role DEFAULT 'contributor',
    assigned_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
    PRIMARY KEY (repository_id, user_id)
);

CREATE TABLE regression_runs (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    repository_id UUID REFERENCES git_repositories(id) ON DELETE CASCADE,
    commit_sha VARCHAR(40) NOT NULL,
    run_state VARCHAR(50) NOT NULL,
    output_log_url TEXT,
```

```
    waveform_artifact_url TEXT,
    initiated_by UUID REFERENCES users(id) ON DELETE SET NULL,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);
```

The normalized tables are mapped to match user progress, workspace edits, and testing metrics:

Relational Table	Primary Keys	Indexes & Relationships	Target Purpose
users	id (UUID)	Unique constraint on username, email	Manages credentials, identity records, and access tokens
user_profiles	user_id (UUID)	Foreign Key -> users.id	Stores candidate profiles, inferred skills, and verified performance levels
challenges	id (UUID)	Unique index on slug	Templates hardware challenges, parameters, and verification requirements[1]
user_submissions	id (UUID)	Foreign Keys -> users.id, challenges.id	Records code submissions, compilation results, and wave file paths[5]
git_repositories	id (UUID)	Foreign Key -> users.id	Tracks collaborative environments, workspaces, and Git configs[14]
repository_access_control	(repository_id, user_id)	Composite pointing to git_repositories, users	Establishes role-based permissions for

			shared workspaces
regression_runs	id (UUID)	Foreign Key -> git_repositories.id	Logs testing pipelines triggered by repository webhooks[15]

Hardware LeetCode Grading Engine

The automated grading engine runs compiles, processes simulations, and evaluates user code asynchronously using sandboxed container runtimes[2].

Execution Orchestrator Integration

Code execution is split into decoupled microservices to handle spikes in submissions smoothly:

1. **Submission Processing**: The gateway receives submitted code and routes it to the Submission Service, which saves the entry as pending and adds a job description to a Redis-backed BullMQ queue[8].
2. **Job Picking**: Dedicated worker nodes retrieve incoming jobs from the queue, pull the challenge assets from object storage, and configure a temporary execution workspace on an SSD-backed volume.
3. **Container Sandbox Execution**: The worker launches a Docker container to run the simulation. The container mounts the workspace as read-only and is restricted by gVisor to prevent access to the host kernel[10].
4. **Log Parsing**: The system monitors simulation standard streams, capturing errors, compile warnings, and generated VCD files[5].
5. **State Processing**: The output log data and generated waveforms are saved to S3. The Submission Service updates PostgreSQL and notifies the client over WebSockets, displaying output logs and rendering waveform traces[4].

```
// submission-producer.js
// Queues incoming user submissions for asynchronous compilation and simulation

const { Queue } = require('bullmq');
const Redis = require('ioredis');

const REDIS_CONNECTION_URI = process.env.REDIS_URL || 'redis://127.0.0.1:6379';
const redisConnection = new Redis(REDIS_CONNECTION_URI, { maxRetriesPerRequest: null
```

```
});

const submissionQueue = new Queue('simulation-tasks', {
  connection: redisConnection,
  defaultJobOptions: {
    attempts: 3,
    backoff: { type: 'exponential', delay: 1000 },
    removeOnComplete: true,
    removeOnFail: false
  }
});

async function publishSubmission                                    {
  const job = await submissionQueue.add('evaluate-verilog', {
    userId,
    challengeId,
    codePayload,
    submittedAt: new Date().toISOString()
  });
  return job.id;
}

module.exports = { publishSubmission };
```

```
// submission-worker.js
// Background task worker running compiles and tests in sandboxed containers

const { Worker } = require('bullmq');
const Redis = require('ioredis');
const { exec } = require('child_process');
const fs = require('fs-extra');
const path = require('path');
const { Pool } = require('pg');

const pgPool = new Pool({ connectionString: process.env.DATABASE_URL });
const redisConnection = new Redis(process.env.REDIS_URL || 'redis://127.0.0.1:6379', {
```

```
maxRetriesPerRequest: null });

const worker = new Worker('simulation-tasks', async (job) => {
  const { userId, challengeId, codePayload } = job.data;
  const executionId = job.id;
  const sandboxPath = path.join('/tmp/sandboxes', executionId);

  await fs.ensureDir(sandboxPath);

  try {
    // Fetch challenge parameters and testbench files
    const dbResult = await pgPool.query(
      'SELECT slug, hidden_testbench_id, execution_timeout_ms, allocation_memory_mb FROM
challenges WHERE id = $1',
      [challengeId]
    );
    if (dbResult.rows.length === 0) throw new Error('Challenge not found');
    const challenge = dbResult.rows[0];

    // Write the user code and reference testbench to the workspace
    const rtlFilePath = path.join(sandboxPath, 'design.sv');
    const tbFilePath = path.join(sandboxPath, 'tb.sv');
    await fs.writeFile(rtlFilePath, codePayload);

    const testbenchCode = await getStoredTestbench(challenge.hidden_testbench_id);
    await fs.writeFile(tbFilePath, testbenchCode);

    // Run simulation within container sandbox
    const evaluation = await runSandboxedExecution(sandboxPath,
challenge.execution_timeout_ms, challenge.allocation_memory_mb);

    // Write evaluation results to database
    await pgPool.query(
      'UPDATE user_submissions SET status = $1, simulation_logs = $2, cpu_run_time_ms = $3
WHERE id = $4',
      [evaluation.status, evaluation.logs, evaluation.runTime, executionId]
    );
  } catch (error) {
    console.error(`Orchestration error on job ${executionId}:`, error.message);
    throw error;
  } finally {
    await fs.remove(sandboxPath);
  }
```

```
}, { connection: redisConnection, concurrency: 8 });

function runSandboxedExecution                                    {
  return new Promise(                {
    const outputLogsPath = path.join(dirPath, 'logs');
    fs.ensureDirSync(outputLogsPath);

    const containerCommand = `docker run --rm --runtime=runsc \
      --network=none \
      --memory="${memoryLimitMb}m" \
      --cpus="1.0" \
      -v ${dirPath}:/workspace:ro \
      -v ${dirPath}/logs:/workspace/logs:rw \
      hdl-compiler-suite \
      sh -c "iverilog -g2012 -o /workspace/logs/sim.out /workspace/design.sv /workspace/tb.sv &&
vvp /workspace/logs/sim.out"`;

    const startTime = Date.now();
    exec(containerCommand, { timeout: timeoutMs },                    {
      const runTime = Date.now() - startTime;
      const logDump = stdout.toString() + stderr.toString();

      if (error) {
        if (error.killed) {
          resolve({ status: 'timeout', logs: 'Execution timed out.', runTime });
        } else {
          resolve({ status: 'compilation_error', logs: logDump, runTime });
        }
      } else {
        const isPassed = logDump.includes('TEST PASSED') &&
!logDump.includes('assertion_error');
        resolve({
          status: isPassed ? 'completed' : 'failed',
          logs: logDump,
          runTime
        });
      }
    });
  });
}

async function getStoredTestbench                 {
  // In production, fetch verification code from secure object storage
```

```
    return `
      module tb();
        reg clk;
        // Testbench code goes here...
      endmodule
    `;
}
```

Sandbox Execution Environment

Standard Docker containers share the host Linux kernel directly, which presents a security risk when executing untrusted, user-submitted code[11]. A kernel vulnerability exploit in one container could compromise the host node or allow unauthorized access to neighboring workspaces[12]. To isolate these processes, the execution engine uses **gVisor** (runsc)[10]. gVisor acts as a user-space kernel that intercepts container system calls through an isolation layer called the "Sentry"[10]. System calls are handled safely in user space instead of being passed directly to the host kernel, shielding the underlying operating system from exploit attempts[10].

```
# Dockerfile: hdl-compiler-suite
# Bundles open-source compilers, synthesis engines, and test frameworks in a hardened environment

FROM ubuntu:22.04

ENV DEBIAN_FRONTEND=noninteractive
ENV PATH="/usr/lib/ccache:/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin"

# Install essential build and compilation dependencies
RUN
```

```
# Install Open-Source Compilation & Synthesis Tools
# 1. Icarus Verilog (IEEE-1364 compilation engine)
RUN

# 2. Verilator (C++ hardware simulation compiler)
RUN

# 3. GHDL (IEEE-1076 VHDL compilation engine)
RUN

# 4. Cocotb (Python-based testbench verification framework)
RUN

# Setup locked-down execution context
RUN
USER sandboxed_user

WORKDIR
```

Integration of KnowAashish/chipdev.io Problem Corpus

The KnowAashish/chipdev.io repository provides 30 structured hardware challenges[1]. Each problem consists of an RTL design template (_rtl.sv) and a corresponding testbench file (_tb.sv)[1].

Challenge Taxonomy & Verification Patterns

The integrated catalog is organized by functional design patterns and specific verification checks:

Code ID	Challenge Title	Core Concept Classification	Verification Mechanics	S3 Testbench Path

Q01	Simple Router	Bus Interconnect routing logic	Cycle-by-cycle routing path assertions[1]	s3://tb-bucket/Q01_Simple_Router_tb.sv [cite: 1]
Q02	Second Largest	Combinational optimization	Multi-pattern data array value checks[1]	s3://tb-bucket/Q02_Second_Largest_tb.sv [cite: 1]
Q03	Rounding Division	Fixed-point arithmetic scaling	Multi-width shift overflow evaluations[1]	s3://tb-bucket/Q03_Rounding_Division_tb.sv [cite: 1]
Q04	Gray Code Counter	Synchronous sequential FSM	Sequential Hamming distance checking[1]	s3://tb-bucket/Q04_Gray_Code_Counter_tb.sv [cite: 1]
Q05	Reversing Bits	Dynamic vector re-mapping	Reverse bit-vector extraction checking[1]	s3://tb-bucket/Q05_Reversing_Bits_tb.sv [cite: 1]
Q06	Edge Detector	State machine transition logic	Synchronous signal transition trace checks[1]	s3://tb-bucket/Q06_Edge_Detector_tb.sv [cite: 1]
Q07	PISO Register	Parallel-to-serial shift conversion	Serial bit-stream shift checks[1]	s3://tb-bucket/Q07_PISO_Shift_Register_tb.sv [cite: 1]
Q08	SIPO Register	Serial-to-parallel conversion	Parallel data register write checks[1]	s3://tb-bucket/Q08_SIPO_Shift_Register_tb.sv [cite: 1]

Q09	Fibonacci Gen	Sequential number generator	State value transitions checking[1]	s3://tb-bucket/Q09_Fibonacci_Generator_tb.sv [cite: 1]
Q10	Counting Ones	Parallel reduction trees	Vector bit count validation[1]	s3://tb-bucket/Q10_Counting_Ones_tb.sv [cite: 1]
Q14	Stopwatch Timer	Real-time clock dividers	Time-limit tracking and cycle checks[1]	s3://tb-bucket/Q14_Stopwatch_Timer_tb.sv [cite: 1]
Q15	Sequence Detector	Overlapping FSM pattern matching	Multi-pattern sequence checks[1]	s3://tb-bucket/Q15_Sequence_Detector_tb.sv [cite: 1]
Q25	Flip Flop Array	Synchronous RAM models	Concurrent read/write checks[1]	s3://tb-bucket/Q25_Flip_Flop_Array_tb.sv [cite: 1, 13]
Q26	Multi-Bit FIFO	Elastic queue structures	Boundary checks for queue states[1]	s3://tb-bucket/Q26_Multi_Bit_FIFO_tb.sv [cite: 1]
Q31	2R1W Register File	Tri-port RAM operations	Simultaneous multi-port access checks[1]	s3://tb-bucket/Q31_2Read1Write_Register_File_tb.sv [cite: 1]

Practical Code Implementations

The implementation of synthesizable logic and its validation testbench is illustrated through the following two examples.

Example 1: Q03 - Rounding Division RTL

This challenge requires a hardware module that calculates:

$$\text{dout} = \left\lfloor \frac{\text{din}}{2^{\text{DIV\_LOG2}}} + 0.5 \right\rfloor$$

To keep the code synthesizable across target FPGA devices, it must avoid using division (/) or modulo (%) operators[20]. Instead, the design uses arithmetic right shifts and extracts remainder bits directly from the lower range of the input vector[20].

```
// Q03_Rounding_Division_rtl.sv
// Implements synthesizable rounding division using bit-shifting operations

module rounding_division #(
   parameter int IN_WIDTH = 16,
   parameter int DIV_LOG2 = 4,
   parameter int OUT_WIDTH = 16
)(
   input  logic [IN_WIDTH-1:0]  din,
   output logic [OUT_WIDTH-1:0] dout
);

   logic [IN_WIDTH-1:0]  shifted_val;
   logic [DIV_LOG2-1:0]  remainder;
   logic [OUT_WIDTH-1:0] raw_out;

   always_comb begin
      // Perform division using arithmetic right shift
      shifted_val = din >> DIV_LOG2;

      // Extract remainder from the lower bit slice
      remainder = din[DIV_LOG2-1:0];

      // Evaluate rounding condition: if remainder is >= 0.5 of divisor
      // Divisor value is 2^(DIV_LOG2), so half divisor value is 2^(DIV_LOG2 - 1)
      if (remainder >= (1 << (DIV_LOG2 - 1))) begin
         raw_out = shifted_val + 1'b1;
```

```
        // Check for potential overflow of outputs
        if (raw_out == 0) begin
          dout = {OUT_WIDTH{1'b1}}; // Saturate on overflow
        end else begin
          dout = raw_out;
        end
      end else begin
        dout = shifted_val[OUT_WIDTH-1:0];
      end
    end

endmodule
```

Example 2: Q25 - Flip Flop Array Testbench

The Flip Flop Array testbench verifies register transitions, write-enable operations, and error handling when simultaneous read and write requests conflict[13].

```
// Q25_Flip_Flop_Array_tb.sv
// Testbench verifying register write-enable transitions and access conflicts

module flip_flop_array_tb();

  bit CLK;
  reg [7:0] DIN;
  reg [2:0] ADDR;
  reg WR;
  reg RD;
  reg RESETN;
  wire [7:0] DOUT;
  wire ERROR;

  // Generate constant clock cycle: 10ns time periods
  always #5 CLK = !CLK;

  // Instantiate Device Under Test (DUT)
```

```
  flip_flop_array DUT (
    .din(DIN),
    .addr(ADDR),
    .wr(WR),
    .rd(RD),
    .clk(CLK),
    .resetn(RESETN),
    .dout(DOUT),
    .error(ERROR)
  );

  // Reset sequence driver task
  task reset();
    RESETN = 0;
    @(posedge CLK);
    RESETN = 1;
  endtask

  // Signal stimulus application task
  task stimulus(
    input bit [7:0] val_din,
    input bit [2:0] val_addr,
    input bit val_wr,
    input bit val_rd
  );
    DIN  = val_din;
    ADDR = val_addr;
    WR   = val_wr;
    RD   = val_rd;
    @(posedge CLK);
  endtask

  initial begin
    // Initialize default register values
    DIN = 0; ADDR = 0; RD = 0; WR = 0;

    // Trace state metrics to stdout
    $monitor("at time t=%0t, RESETN=%0b DIN=%0b ADDR=%0b WR=%0b RD=%0b
DOUT=%0b ERROR=%0b",
          $time, RESETN, DIN, ADDR, WR, RD, DOUT, ERROR);

    // Run Verification Test Cases
    reset();
```

```
    // Test 1: Sequence two distinct writes, then read them back
    stimulus(8'hFF, 3'd1, 1'b1, 1'b0); // Write Address 1
    stimulus(8'hEE, 3'd2, 1'b1, 1'b0); // Write Address 2
    stimulus(8'h00, 3'd1, 1'b0, 1'b1); // Read Address 1 (Expect 8'hFF)
    stimulus(8'h00, 3'd2, 1'b0, 1'b1); // Read Address 2 (Expect 8'hEE)

    // Test 2: Raise error assert signal by running concurrent Read and Write operations
    stimulus(8'hAA, 3'd3, 1'b1, 1'b1);

    #50;
    $finish;
  end

  // Direct Waveform Generation Outputs
  initial begin
    $dumpfile("outputs/dump.vcd");
    $dumpvars(0, flip_flop_array_tb);
  end

endmodule
```

Collaborative Cloud Workspaces and Embedded Git Server

The system architecture supports shared workspace projects, managed programmatically through self-hosted Git services[21].

Repository Provisioning via Gitea API

To manage user workspaces, the system orchestrates a self-hosted Gitea service using Gitea's REST API[15]. The Project Service automatically provisions private repositories and configures webhooks to trigger testing pipelines when commits are pushed[15].

```
// repository-provisioner.js
// Handles programmatic Gitea repository creation and webhook configuration
```

```
const axios = require('axios');

const GITEA_API_ENDPOINT = process.env.GITEA_API_URL || 'http://gitea-srv:3000/api/v1';
const ADMIN_OAUTH_TOKEN = process.env.GITEA_ADMIN_TOKEN;

async function setupProjectRepository                                    {
  const creationUrl = `${GITEA_API_ENDPOINT}/orgs/${teamSlug}/repos`;

  const repoPayload = {
    name: projectName,
    description: `Automated repository backing workspace: ${projectName}`,
    private: true,
    auto_init: true,
    gitignores: "Verilog",
    readme: "Default"
  };

  try {
    const response = await axios.post(creationUrl, repoPayload, {
      headers: {
        'Content-Type': 'application/json',
        'Authorization': `token ${ADMIN_OAUTH_TOKEN}`
      }
    });
    return {
      cloneUrl: response.data.clone_url,
      sshUrl: response.data.ssh_url,
      fullName: response.data.full_name
    };
  } catch (err) {
    console.error('Failed to provision Gitea repository:', err.response?.data || err.message);
    throw new Error('Repository creation failed.');
  }
}

async function attachPipelineWebhook                                                {
  const webhookUrl = `${GITEA_API_ENDPOINT}/repos/${repoFullName}/hooks`;

  const webhookPayload = {
    type: 'gitea',
    active: true,
    events: ['push'],
    config: {
```

```
            content_type: 'json',
            url: callbackWebhookUrl
        }
    };

    try {
        const response = await axios.post(webhookUrl, webhookPayload, {
            headers: {
                'Content-Type': 'application/json',
                'Authorization': `token ${ADMIN_OAUTH_TOKEN}`
            }
        });
        return response.data.id;
    } catch (err) {
        console.error('Failed to configure pipeline webhook:', err.response?.data || err.message);
        throw new Error('Webhook registration failed.');
    }
}

module.exports = { setupProjectRepository, attachPipelineWebhook };
```

Monaco Editor Language Server Protocol (LSP) Configuration

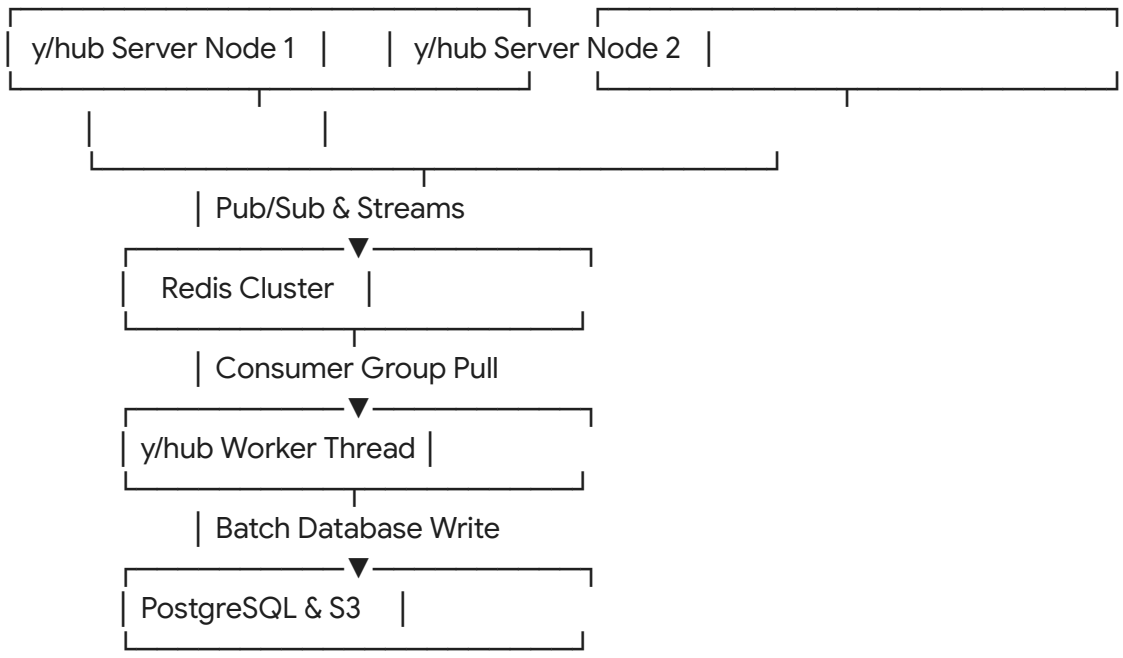
The web editor connects the Monaco text engine to a containerized svls (SystemVerilog Language Server) process over WebSockets to provide autocomplete, formatting, and inline syntax checking[23].

```
// MonacoLanguageClientConnector.ts
// Configures Monaco to connect with a remote SystemVerilog Language Server over WebSockets

import { Monaco } from '@monaco-editor/react';
import { MonacoLanguageClient, MessageTransports } from 'monaco-languageclient';
import { toSocket, WebSocketMessageReader, WebSocketMessageWriter } from
'vscode-ws-jsonrpc';

export function connectLanguageServer                                    any
string  {
```

```
  // 1. Register SystemVerilog language rules in Monaco [cite: 27, 28]
  monaco.languages.register({
    id: 'systemverilog',
    extensions: ['.sv', '.svh'],
    aliases: ['SystemVerilog', 'sv']
  });

  // 2. Establish connection to remote Language Server socket [cite: 26, 28]
  const socketUrl = `${process.env.NEXT_PUBLIC_LSP_SOCKET_URL || 'ws://localhost:3015'}/svls`;
  const webSocket = new WebSocket(socketUrl);

  webSocket.onopen =        {
    const socket = toSocket(webSocket);
    const reader = new WebSocketMessageReader(socket);
    const writer = new WebSocketMessageWriter(socket);

    const languageClient = new MonacoLanguageClient({
      name: 'SystemVerilog Language Client',
      clientOptions: {
        documentSelector: ['systemverilog'],
        errorHandler: {
          error:        ({ action: 1 }), // Continue running on error [cite: 28, 29]
          closed:         ({ action: 2 })  // Do not restart closed sessions [cite: 28, 29]
        }
      },
      connectionProvider: {
        get:        Promise.resolve({ reader, writer } as MessageTransports)
      }
    });

    languageClient.start();

    // Bind the current document path to the LSP session context [cite: 28]
    const currentModel = editorInstance.getModel();
    if (currentModel) {
      monaco.editor.setModelLanguage(currentModel, 'systemverilog');
    }
  };

  webSocket.onerror =             {
    console.error('LSP WebSocket connection error:', err);
  };
}
```

Clustered Yjs Collaboration Backend

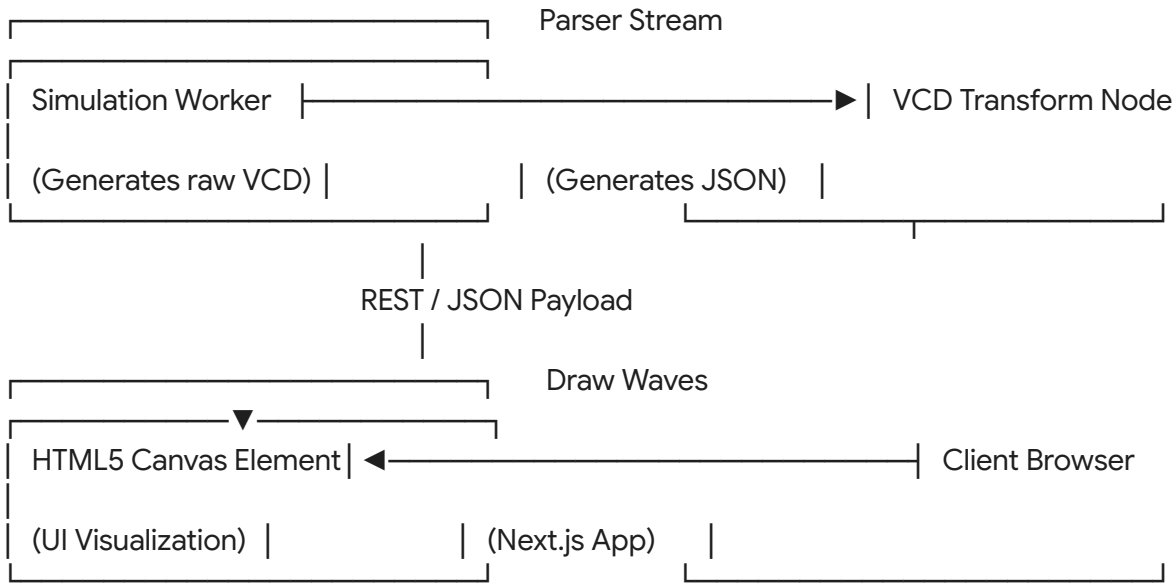
Real-time collaboration uses standard Yjs CRDTs to synchronize document changes across users without conflict[4]. In production, the system uses a clustered y/hub deployment[6]. This setup uses Redis Pub/Sub channels to sync updates across editor nodes and coordinates database writes to S3 and PostgreSQL through background workers[6].



When a user edits a shared document, changes are processed as Yjs updates and sent over WebSockets to the active server node[4]. This node broadcasts the updates to all other clients connected to the same session via Redis[6]. The database worker processes modifications through a consumer group, periodically merging update logs and saving document states to permanent storage[6].

Unified Waveform Visualization Engine

Visualizing digital timing and transition patterns requires a web-native waveform rendering tool. The system processes raw Value Change Dump (VCD) files, converts them into a lightweight JSON structure, and draws the traces in the browser using an HTML5 Canvas component[5].



The system translates VCD transitions to coordinate mappings, drawing single-bit clock steps and multi-bit data buses directly inside the browser canvas[17].

Web-Native VCD Signal Processing Engine

During simulation runs, the grading engine converts raw, text-heavy VCD files into structured, index-aligned JSON schemas[5].

```
// vcd-parser.js
// Parses IEEE 1364 Value Change Dump (VCD) logs into structured JSON payloads [cite: 5, 32]

const fs = require('fs');
const readline = require('readline');

async function processVcdFile                {
  const fileStream = fs.createReadStream(inputPath);
  const lineReader = readline.createInterface({ input: fileStream, crlfDelay: Infinity });
```

```
const traceData = {
  timescale: '1ns',
  endtime: 0,
  signals: {}
};

let activeTimestamp = 0;
const registerMap = new Map();

for await (const rawLine of lineReader) {
  const line = rawLine.trim();
  if (!line) continue;

  // Parse VCD scale and header metadata
  if (line.startsWith('$timescale')) {
    traceData.timescale = line.split(' ').slice(1, 3).join('');
    continue;
  }

  // Map signal symbols to variable definitions
  if (line.startsWith('$var')) {
    const parts = line.split(/\s+/);
    const varType = parts[1];
    const bitWidth = parseInt(parts[2], 10);
    const refChar = parts[3];
    const signalName = parts[4];

    registerMap.set(refChar, signalName);
    traceData.signals[signalName] = {
      name: signalName,
      type: varType,
      size: bitWidth,
      wave: [] // Dynamic timeline tuples: [timestamp, state_val]
    };
    continue;
  }

  // Parse simulation time steps
  if (line.startsWith('#')) {
    activeTimestamp = parseInt(line.substring(1), 10);
    if (activeTimestamp > traceData.endtime) {
      traceData.endtime = activeTimestamp;
```

```
            }
            continue;
        }

        // Parse single-bit state transitions
        if (line.match(/^[01zZxX]/)) {
            const stateValue = line[0];
            const refChar = line.substring(1);
            const signalName = registerMap.get(refChar);
            if (signalName) {
                traceData.signals[signalName].wave.push([activeTimestamp, stateValue]);
            }
            continue;
        }

        // Parse multi-bit vector transitions
        if (line.startsWith('b') || line.startsWith('B')) {
            const parts = line.split(/\s+/);
            const vectorValue = parts[0].substring(1);
            const refChar = parts[1];
            const signalName = registerMap.get(refChar);
            if (signalName) {
                traceData.signals[signalName].wave.push([activeTimestamp, vectorValue]);
            }
        }
    }

    return traceData;
}

module.exports = { processVcdFile };
```

This service produces a compressed JSON payload mapping the timestamp of every state transition:

```
{
  "timescale": "1ns",
```

```
  "endtime": 120,
  "signals": {
    "clk": {
      "name": "clk",
      "type": "reg",
      "size": 1,
      "wave": [[0, "0"], [5, "1"], [10, "0"], [15, "1"], [20, "0"]]
    },
    "dout": {
      "name": "dout",
      "type": "wire",
      "size": 8,
      "wave": [[0, "x"], [10, "FF"], [20, "EE"]]
    }
  }
}
```

This JSON output is processed by a HTML5 Canvas component in the frontend client. This component maps the state transitions to coordinate matrices and renders the digital waves dynamically:

```
// CanvasWaveformRenderer.tsx
// Renders digital signal steps and buses using an HTML5 Canvas interface

import React, { useEffect, useRef } from 'react';

interface SignalTimeline {
  name: string;
  type: string;
  size: number;
  wave: [number, string][];
}

interface CanvasWaveformRendererProps {
  parsedVcd: {
    timescale: string;
    endtime: number;
```

```
    signals: Record<string, SignalTimeline>;
  };
}

export const CanvasWaveformRenderer: React.FC<CanvasWaveformRendererProps> =
                    {
  const canvasElementRef = useRef<HTMLCanvasElement | null>(null);

  useEffect(      {
    const canvas = canvasElementRef.current;
    if (!canvas) return;

    const ctx = canvas.getContext('2d');
    if (!ctx) return;

    const pixelsPerTimeStep = 6;
    const rowHeight = 50;
    const startXOffset = 100;
    const signalsArray = Object.values(parsedVcd.signals);

    // Adjust canvas dimensions dynamically to prevent scaling artifacts
    canvas.width = startXOffset + (parsedVcd.endtime * pixelsPerTimeStep) + 50;
    canvas.height = (signalsArray.length * rowHeight) + 60;

    // Apply background theme
    ctx.fillStyle = '#0f172a'; // slate-900 editor background color
    ctx.fillRect(0, 0, canvas.width, canvas.height);

    // Draw timescale division lines
    ctx.strokeStyle = '#334155'; // slate-700 guide line color
    ctx.lineWidth = 0.5;
    for (let timeStep = 0; timeStep <= parsedVcd.endtime; timeStep += 10) {
      const xCoord = startXOffset + (timeStep * pixelsPerTimeStep);
      ctx.beginPath();
      ctx.moveTo(xCoord, 30);
      ctx.lineTo(xCoord, canvas.height - 30);
      ctx.stroke();

      ctx.fillStyle = '#64748b'; // slate-500 scale label font
      ctx.font = '9px monospace';
      ctx.fillText(`${timeStep}ns`, xCoord - 10, 20);
    }
```

```
// Draw signal waves
signalsArray.forEach(                                {
  const yBaseOffset = (rowIndex * rowHeight) + 40;

  // Draw signal name labels
  ctx.fillStyle = '#f1f5f9'; // slate-100 label text
  ctx.font = 'bold 11px monospace';
  ctx.fillText(`${signal.name}`, 15, yBaseOffset + (rowHeight / 2) + 4);

  ctx.strokeStyle = '#10b981'; // emerald-500 waveform trace color
  ctx.lineWidth = 2.0;
  ctx.beginPath();

  let lastX = startXOffset;
  let lastY = yBaseOffset + (signal.size === 1 ? rowHeight - 15 : rowHeight - 10);

  signal.wave.forEach(                                        {
    const nextX = startXOffset + (timeStep * pixelsPerTimeStep);

    if (signal.size === 1) {
      // For single-bit clock/control lines, render square waves
      const nextY = yBaseOffset + (val === '1' ? 10 : rowHeight - 15);
      if (waveIndex > 0) {
        ctx.lineTo(nextX, lastY); // Vertical step transitions
      }
      ctx.lineTo(nextX, nextY); // Horizontal signal hold steps
      lastY = nextY;
    } else {
      // For multi-bit buses, render hexagonal data bus shapes
      ctx.lineTo(nextX - 3, lastY);
      ctx.lineTo(nextX, yBaseOffset + (rowHeight / 2));
      ctx.lineTo(nextX + 3, lastY);

      // Render string values inside the bus boundaries
      ctx.fillStyle = '#0ea5e9'; // sky-500 trace value data hex
      ctx.font = '9px monospace';
      ctx.fillText(val, nextX + 12, yBaseOffset + (rowHeight / 2) + 4);
    }
    lastX = nextX;
  });

  // Extend the final wave state to simulation limit
  const finalX = startXOffset + (parsedVcd.endtime * pixelsPerTimeStep);
```

```
            ctx.lineTo(finalX, lastY);
            ctx.stroke();
        });
    }, [parsedVcd]);

    return (
        <div className="w-full overflow-x-auto bg-slate-900 p-4 border border-slate-800
rounded-lg">
            <canvas ref={canvasElementRef} className="mx-auto" />
        </div>
    );
};
```

Unified Portfolio and Recruiter Dashboard

The platform gathers and aggregates coding submissions and repository metrics to compute a verified skill score for candidates.

Skill Scoring Algorithm

The candidate evaluation model uses mathematical formulations to generate a normalized **Role Readiness Score** ($S_{\text{readiness}}$). The model factors in challenge difficulty weights, solution runtimes, and repository contributions:

$$S_{\text{readiness}} = \alpha \cdot \left(\sum_{i \in \text{Completed}} D_i \cdot T_i \cdot M_i \right) + \beta \cdot \left(\sum_{k \in \text{Repos}} C_k \cdot \ln(1 + A_k) \right)$$

The variables represent the following parameters:

- D_i : The assigned weight score of challenge difficulty i ($\text{Easy} = 1.0$, $\text{Medium} = 2.5$, $\text{Hard} = 5.0$).
- T_i : The submission efficiency score:

$$T_i = \min\left(1.0, \frac{\text{TimeoutLimit}_i}{\text{UserRunTime}_i}\right)$$

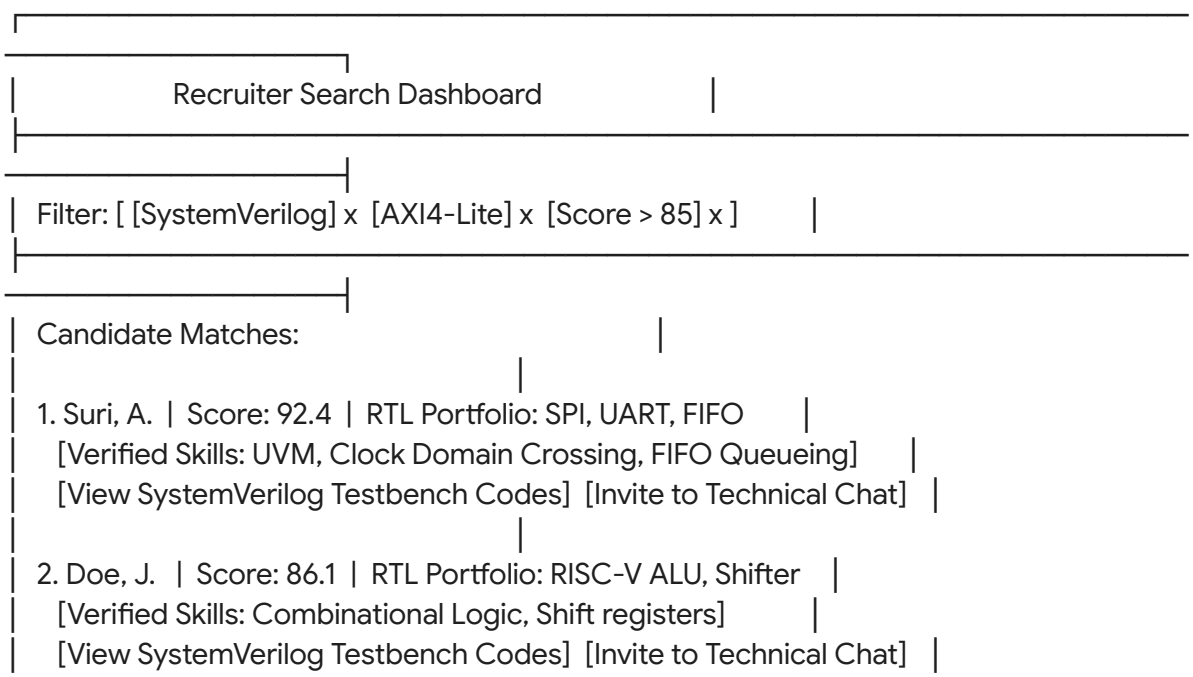
- M_i : The test completeness score, determined by the ratio of passed test assertions:

$$M_i = \frac{\text{AssertionsPassed}_i}{\text{TotalAssertions}_i}$$

- C_k : The workspace contribution multiplier, reflecting the user's role on project k ($\text{Owner} = 1.2$, $\text{Maintainer} = 1.0$, $\text{Contributor} = 0.8$, $\text{Viewer} = 0.0$).
- A_k : The number of verified commits pushed to remote repository branch files on project k.
- α, β : Scaling constants used to normalize the output score.

Recruiter Analytics View

The Recruiter Dashboard enables employers to discover candidates using searchable talent profiles. These profiles are generated automatically based on verified challenge submissions and repository contributions[2]:



The matching index evaluates specific keywords in the user's code history (such as always_ff, always_comb, assert property, and uvm_component) to verify skill competencies directly. This search engine gives employers transparent, verifiable insights into candidates' hands-on hardware engineering skills.

Workspace Pre-Commit CI & Testing Pipeline

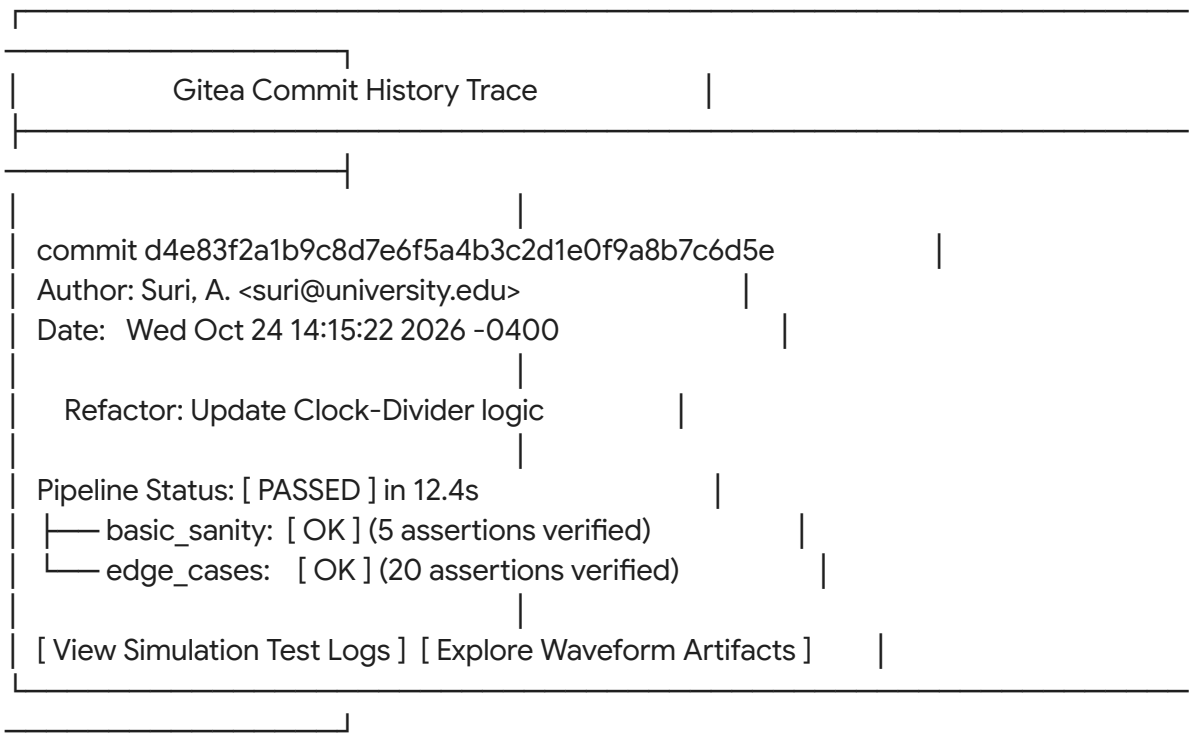
To maintain code quality across collaborative projects, every repository update automatically triggers a verification pipeline[15]. This pipeline runs regression tests, compiles updated modules, and logs test results[15].

The build sequence is configured via a manifest file named bitforbytes.yml, which is placed in the root of each repository:

```yaml
# bitforbytes.yml
# Configuration schema defining automated testing pipelines for hardware projects

pipeline_profile:
  target_toolchain: verilator # Supported compilation profiles: [iverilog, verilator, ghdl] [cite: 33, 34]
  workspace_entry: design.sv
  top_level_testbench: tb.sv
  compilation_flags:
    - -Wall
    - --trace
    - --timing
  execution_limits:
    timeout_seconds: 30
    memory_allocation_mb: 512

regression_matrix:
  test_suites:
    - name: basic_sanity
      command: iverilog -g2012 -o test_basic design.sv tb.sv && vvp test_basic
      required_asserts: 5
    - name: edge_cases
      command: verilator --binary -j 4 --trace --top-module tb design.sv tb.sv && ./obj_dir/Vtb
```

required_asserts: 20

When a user pushes changes to Gitea, Gitea executes a POST request to the pipeline orchestrator[15]. The orchestrator extracts the repository name and commit hash, creates a testing workspace, reads the bitforbytes.yml manifest, and queues verification jobs[8]. Once processing is complete, the testing engine writes validation metrics directly to the git commit history:



This workflow ensures that users write and verify designs in a controlled continuous integration loop, mirroring modern professional hardware development pipelines.

Production Deployment and Integration Roadmap

Integrating these systems into production requires a phased rollout. This roadmap details the engineering timeline, key architectural tasks, systems dependencies, and testing gates for each phase:

Phase Name	Timeline	Core	Backend API	Verification

		Infrastructure Focus	Integration Tasks	Gates & Testing Metrics
Phase 1: Grading Engine	Months 1-3	Sandbox execution pools, Redis BullMQ runner nodes, Next.js UI elements[8]	Install gVisor isolation runtimes, package compilation suites, write execution queue brokers[8]	Ensure sub-500ms execution latency under concurrency, validate system call isolation, test testbench auto-grading
Phase 2: Collab Workspaces	Months 4-6	Programmatic Gitea APIs, clustered Yjs providers, custom VCD parsers[5]	Build Gitea integration handlers, connect clustered WebSocket providers, deploy Canvas rendering pipelines[6]	Test collaboration updates under high network latency, verify edit sync correctness, test parsing speed on large VCD trace files
Phase 3: Portfolio & Ecosystem	Months 7-9	Scoring indices, lesson pathways, search indexers	Implement the role readiness scoring engine, configure step-by-step problem pathways, launch recruiter portals	Verify score normalization patterns, test RBAC filter policies, optimize database query latency for complex user profiles
Phase 4: ASIC Synthesizer	Months 10+	Yosys compiler tools, nextpnr routing logic[33]	Connect open-source FPGA synthesis	Validate routing layout exports, audit container

			tooling, launch routing visualizations[33]	resource usage, evaluate compilation costs under peak server loads

Architectural Conclusions

The unified system design of this platform addresses the key educational and infrastructure challenges of hardware engineering validation. By separating core microservices, isolating compilation tasks with gVisor, and evaluating submissions using cycle-accurate pin assertions, the platform executes untrusted user code safely and efficiently[2].

Additionally, combining Web-based Monaco Editor workspaces, real-time Yjs collaboration, self-hosted Gitea clusters, and high-performance VCD canvas wave renderers gives users a powerful online hardware development environment[6]. This integrated architecture bridges the gap between hardware verification concepts and professional engineering workflows, providing a robust platform for both learning and evaluation.

Works cited

1. This repository consists of all chipdev.io practice questions · GitHub, https://github.com/KnowAashish/chipdev.io
2. SystemVerilog coding and simulation website, aimed at interview prep : r/ECE - Reddit, https://www.reddit.com/r/ECE/comments/q7phdc/systemverilog_coding_and_simulation_website_aimed/
3. Ashish Suri KnowAashish - GitHub, https://github.com/KnowAashish
4. Yjs WebSocket Server Guide: Setup & Scaling in 2025 | Velt, https://velt.dev/blog/yjs-websocket-server-real-time-collaboration
5. ahmed-agiza/vcd-parser: VCD Parser for Node.js - GitHub, https://github.com/ahmed-agiza/vcd-parser
6. GitHub - yjs/yhub: Alternative backend for y-websocket, https://github.com/yjs/yhub
7. Y-redis: An alternative backend to y-websocket - Show - Yjs Community, https://discuss.yjs.dev/t/y-redis-an-alternative-backend-to-y-websocket/2509
8. How to Build a Job Queue in Node.js with BullMQ and Redis - OneUptime, https://oneuptime.com/blog/post/2026-01-06-nodejs-job-queue-bullmq-redis/view
9. Trigger.dev vs BullMQ, https://trigger.dev/vs/bullmq
10. Introduction to gVisor security, https://gvisor.dev/docs/architecture_guide/intro/
11. How to run AI-generated code safely | Blog - Northflank, https://northflank.com/blog/run-ai-generated-code

12. Use GKE Sandbox gVisor to Isolate Untrusted Workloads at the Container Level, https://oneuptime.com/blog/post/2026-02-17-how-to-use-gke-sandbox-gvisor-to-isolate-untrusted-workloads-at-the-container-level/view
13. chipdev.io/Q25_Flip_Flop_Array_tb.sv at main · KnowAashish/chipdev.io · GitHub, https://github.com/KnowAashish/chipdev.io/blob/main/Q25_Flip_Flop_Array_tb.sv
14. Howto create repository using api - API/SDK/Tea/HelmChart/Terraform - Gitea, https://forum.gitea.com/t/howto-create-repository-using-api/3821
15. Gitea webhooks never created — OAuth scopes missing write:repository #4412 - GitHub, https://github.com/Dokploy/dokploy/issues/4412
16. Webhooks - Gitea, https://go-gitea-gitea.mintlify.app/integrations/webhooks
17. A cloud based signal waveform viewer using modern browser technologies., http://mocast.physics.auth.gr/images/NewPapers/PAPER_15F.pdf
18. Security Model - gVisor, https://gvisor.dev/docs/architecture_guide/security/
19. This repository contains VHDL implementations for all hardware design challenges (quests/questions) featured on the chipdev.io website. - GitHub, https://github.com/nselvara/chipdev.io
20. chipdev.io/Q03_Rounding_Division_rtl.sv at main · KnowAashish/chipdev.io · GitHub, https://github.com/KnowAashish/chipdev.io/blob/main/Q03_Rounding_Division_rtl.sv
21. gitea | Skills Marketplace - LobeHub, https://lobehub.com/skills/membranedev-application-skills-gitea
22. How to Use Ansible to Create Git Repositories - OneUptime, https://oneuptime.com/blog/post/2026-02-21-how-to-use-ansible-to-create-git-repositories/view
23. Language Server, https://langserver.org/
24. awesome-stars/README.md at master - GitHub, https://github.com/jiegec/awesome-stars/blob/master/README.md
25. GitHub - TypeFox/monaco-languageclient: A toolbox for building web applications with editors utilizing language servers., https://github.com/typefox/monaco-languageclient
26. monaco-languageclient/docs/guides/troubleshooting.md at main - GitHub, https://github.com/TypeFox/monaco-languageclient/blob/main/docs/guides/troubleshooting.md
27. Yjs - Best of JS, https://bestofjs.org/projects/yjs
28. Self-Hosted Collaboration - SuperDoc, https://docs.superdoc.dev/guides/collaboration/self-hosted-overview
29. YosysHQ/oss-cad-suite-build - GitHub, https://github.com/yosyshq/oss-cad-suite-build